CATEGORY 6

Adversarial Search for Game Playing AI Techniques

Combined Python Program

This menu-driven program contains Min-Max search for a simplified chess decision tree, Alpha-Beta pruning for a simplified chess decision tree, and Min-Max search for Tic-Tac-Toe.

Program

```
def minimax(node, depth, maximizing):

    if depth == 0:
        return node

    if maximizing:
        best = -999
        for child in node:
            value = minimax(child, depth - 1, False)
            best = max(best, value)
        return best

    else:
        best = 999
        for child in node:
            value = minimax(child, depth - 1, True)
            best = min(best, value)
        return best


def alphabeta(node, depth, alpha, beta, maximizing):

    if depth == 0:
        return node

    if maximizing:
        best = -999
        for child in node:
            value = alphabeta(child, depth - 1,
                              alpha, beta, False)
            best = max(best, value)
            alpha = max(alpha, best)

            if alpha >= beta:
                break
        return best

    else:
        best = 999
        for child in node:
            value = alphabeta(child, depth - 1,
                              alpha, beta, True)
            best = min(best, value)
            beta = min(beta, best)

            if alpha >= beta:
                break
        return best


def winner(board):

    wins = [
        [0,1,2], [3,4,5], [6,7,8],
        [0,3,6], [1,4,7], [2,5,8],
        [0,4,8], [2,4,6]
    ]

    for a,b,c in wins:
        if board[a] == board[b] == board[c] != ' ':
            return board[a]

    return None


def tic_minimax(board, maximizing):

    w = winner(board)
```

```
    if w == 'X':
        return 1
    if w == 'O':
        return -1
    if ' ' not in board:
        return 0

    if maximizing:
        best = -999

        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                score = tic_minimax(board, False)
                board[i] = ' '
                best = max(best, score)

        return best

    else:
        best = 999

        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                score = tic_minimax(board, True)
                board[i] = ' '
                best = min(best, score)

        return best


# Main Program

print("ADVERSARIAL SEARCH")
print("1. Min-Max Chess")
print("2. Alpha-Beta Pruning Chess")
print("3. Min-Max Tic-Tac-Toe")

choice = int(input("Enter your choice: "))

chess = [
    [3, 5],
    [2, 9],
    [4, 6]
]

if choice == 1:

    result = minimax(chess, 2, True)
    print("Best Chess Move Evaluation:", result)

elif choice == 2:

    result = alphabeta(chess, 2, -999, 999, True)
    print("Best Chess Move Evaluation:", result)

elif choice == 3:

    board = [
        'X', 'O', 'X',
        ' ', 'O', ' ',
        ' ', ' ', ' '
    ]

    best_score = -999
    best_move = -1

    for i in range(9):
        if board[i] == ' ':
            board[i] = 'X'
            score = tic_minimax(board, False)
            board[i] = ' '

            if score > best_score:
                best_score = score
                best_move = i

    print("Best Move:", best_move)
    print("Best Score:", best_score)

else:
```

```
    print("Invalid Choice")
```

Sample Outputs

Option 1 – Min-Max Chess

```
ADVERSARIAL SEARCH
1. Min-Max Chess
2. Alpha-Beta Pruning Chess
3. Min-Max Tic-Tac-Toe
Enter your choice: 1
Best Chess Move Evaluation: 5
```

Option 2 – Alpha-Beta Pruning Chess

```
Enter your choice: 2
Best Chess Move Evaluation: 5
```

Option 3 – Min-Max Tic-Tac-Toe

```
Enter your choice: 3
Best Move: 7
Best Score: 1
```

Result

Thus, a combined menu-driven Python program for the three adversarial search techniques was implemented successfully.

Note: The two chess options use simplified decision-tree examples with evaluation scores; they are not complete playable chess engines.